ADITYA UNIVERSITY

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the project work entitled **"Intelligent Loan Risk Assesment System"** is submitted by

Balabhadra Chaitanya Satya Kishore	(24B11CS027)
Sapparapu Teja Naga Satya Ganesh	(24B11CS394)
Kella yugandhar	(24B11CS205)
Addenkila Durga Sumanth	(25B21CS011)

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026–2027.

Project Guide
Mrs.Jyothula Vidya
Assistant Professor
Department of CSE

Head of the Department
Dr A. Phani Sridhar,
Associate Professor & HOD
Department of CSE

DECLARATION

We hereby declare that the project entitled **"INTELLIGENT LOAN RISK ASSESMENT SYSTEM"** is original and is submitted to **Aditya University, Surampalem** in partial fulfilment of the requirements for the award of the B.Tech. degree in **Computer Science and Engineering.** We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem
Date:

By

B.Chaitanya(24B11CS027)

K.Yugandhar(24B11CS205)

S.Teja(24B11CS394)

A.Sumanth(25B21CS011)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. Jyothula Vidya , Assistant Professor,** who has guided us throughout the project. Their valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar K - Associate Professor, Dr. Hari Jyothula-Associate Professor, Dr. Siva Satya Prasad P - Assistant Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department, **Dr. A. Phani Sridhar, Associate Professor & HOD**, for their inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing**, for his encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar; Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences); Dr. S. Rama Sree, Pro Vice-Chancellor (Academics); Dr. M.B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University,** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

ABSTRACT

Loan risk assessment is an important process in the banking and financial sector, as identifying high-risk applicants helps financial institutions make informed lending decisions. This project, **Loan Risk Assessment Using Machine Learning**, uses applicant and financial information to predict whether a loan applicant belongs to the Low Risk or High Risk category. The project uses a dataset containing 1,000 records and 10 columns, including applicant details, monthly income, loan amount, loan term, credit score, previous defaults, and monthly debt payment. Data preprocessing techniques such as missing value handling, duplicate removal, and invalid value treatment were applied to improve data quality. Feature engineering was performed by creating a Debt-to-Income (DTI) Ratio to represent the relationship between monthly debt payments and monthly income. The dataset was divided into 80% training data and 20% testing data using stratified splitting, and SMOTE was applied to the training data to address class imbalance. Three supervised machine learning algorithms—Logistic Regression, Random Forest, and XGBoost—were trained and compared using Accuracy, Precision, Recall, F2-Score, confusion matrices, and ROC-AUC. A Streamlit-based web interface was developed to provide a user-friendly way to enter applicant information and obtain loan risk predictions. The system demonstrates how machine learning can support faster, consistent, and data-driven loan risk assessment. It is intended as a decision-support tool and not as a replacement for professional financial evaluation.

Keywords: Loan Risk Assessment, Machine Learning, Logistic Regression, Random Forest, XGBoost, SMOTE, Debt-to-Income Ratio, Model Evaluation, Streamlit.

LIST OF CONTENTS

LIST OF CONTENTS

Chapter 1: Introduction 01

1.1 Introduction 01

1.2 Problem Statement 02

1.3 Objectives of the Project 02

1.4 Scope of the Project 03

Chapter 2: System Overview 04

2.1 Existing System 04

2.2 Limitations of Existing System 04

2.3 Proposed System 05

2.4 Software Requirements 05

2.5 Hardware Requirements 05

Chapter 3: System Implementation 06

3.1 Overview of Development Tools 06

3.1.1 Visual Studio Code and Python 06

3.1.2 scikit-learn and Supporting Libraries 07

3.2 Dataset Description 07

3.3 Feature Description 08

3.4 Running the Project 10

Chapter 4: System Architecture 11

Chapter 5: Data Preprocessing 14

5.1 Introduction 14

5.2 Handling Missing Values 14

5.3 Data Cleaning 14

5.4 Outlier Detection and Range Validation 14

5.5 Feature Engineering and Encoding 15

5.6 Data Visualization 15

5.7 Train-Test Split and Feature Scaling 15

5.8 Summary 16

Chapter 6: Methodology and Algorithms 17
6.1 Introduction 17
6.2 Logistic Regression 17
6.3 Random Forest 18
6.4 XGBoost 19
6.5 Model Evaluation Metrics 20
6.6 Algorithm Configuration 23
6.7 Training and Saving Procedure 24

Chapter 7: Results and Analysis 26
7.1 Introduction 26
7.2 Experimental Setup 26
7.3 Logistic Regression Results 27
7.4 Random Forest Results 27
7.5 XGBoost Results 28
7.6 Comparison of Models 28
7.7 Feature Importance Analysis 29
7.8 Key Findings 30
7.9 Discussion 31
7.10 Limitations of the Study 32
7.11 Key Findings 33

Chapter 8: Output Screens 34

Chapter 9: Conclusion 37

Chapter 10: Future Scope 38

Chapter 11: References 39

LIST OF FIGURES

Fig. 3.1: VS Code Project Folder Structure and Virtual Environment Setup

Fig. 4.1: Overall Architecture of the Intelligent Loan Risk Assessment System

Fig 6.1: Training the Logistic Regression Model

Fig 6.2: Training the Random Forset Model

Fig 6.3: Training the XGBoost Model

Fig 6.5.1 Confusion Matrix for Machine Learning Models

Fig 6.5.2 ROC-AUC Curve for Evalution of Machine Learning Models

Fig 6.5.3 Evalutaion Metrices Result Image

Fig 7.7 Feature Important Analysis of Three Trained ML Models

Fig. 8.1: Streamlit Application – Applicant Information Input Form

Fig. 8.2: Loan Risk Prediction Result

Fig. 8.3: Monthly Income vs. Loan Amount

Fig. 8.4: Credit Score Distribution

CHAPTER 1
INTRODUCTION

1.1 Introduction

Loan risk assessment is an important process in the banking and financial sector because it helps financial institutions identify applicants who may have difficulty repaying their loans. Traditional loan assessment methods can be time-consuming and may not effectively analyse all the financial factors associated with an applicant. Factors such as age, monthly income, loan amount, loan term, credit score, previous defaults, and monthly debt payments play an important role in determining the financial risk associated with a loan application.

Machine Learning provides an effective approach to analysing financial data and identifying patterns that may be difficult to detect through manual assessment alone. Supervised classification algorithms can learn from historical loan data and use the learned patterns to classify new applicants into different risk categories. Feature engineering techniques, such as calculating the Debt-to-Income (DTI) Ratio, can provide additional information about an applicant's financial obligations relative to their income.

This project, **Intelligent Loan Risk Assessment System**, uses a dataset containing 1,000 records and 10 columns to predict whether a loan applicant belongs to the Low Risk or High Risk category. The project involves data preprocessing, exploratory data analysis, feature selection, DTI feature engineering, and handling class imbalance using the Synthetic Minority Over-sampling Technique (SMOTE). Three Machine Learning algorithms—Logistic Regression, Random Forest, and XGBoost—are trained and compared using different evaluation metrics. A Streamlit-based user interface is also developed to allow users to enter applicant details and obtain a loan risk prediction. The system aims to support faster, consistent, and data-driven preliminary loan risk assessment.

1.2 Problem Statement

Traditional loan assessment methods may require considerable time and manual effort to analyse an applicant's financial information. Assessing loan risk based on individual factors alone may not fully capture the relationships between income, loan amount, credit score, previous defaults, and existing debt obligations. These limitations can make it difficult to identify potentially high-risk applicants consistently.

An intelligent system is therefore needed to analyse multiple financial factors together, identify patterns in historical loan data, and predict the risk category of a loan applicant. The proposed system uses Machine Learning algorithms to classify applicants into Low Risk and High Risk categories, compare the performance of different models, and provide predictions through a simple user interface. This approach can assist in preliminary risk assessment and support informed lending decisions.

1.3 Objectives of the Project

The main objectives of the project are:

• To prepare and clean the loan risk dataset containing **1,000 records and 10 columns**.

• To perform Exploratory Data Analysis (EDA) to understand the distribution and relationships of applicant and financial features.

• To select important features, including age, monthly income, loan amount, loan term, credit score, previous defaults, and monthly debt payment.

• To create a new feature called **Debt-to-Income (DTI) Ratio** to represent the relationship between monthly debt payments and monthly income.

• To divide the dataset into **80% training data and 20% testing data** using stratified splitting.

• To apply **SMOTE** to the training data to address class imbalance.

• To train and compare three Machine Learning algorithms: **Logistic Regression, Random Forest, and XGBoost**.

• To evaluate model performance using Accuracy, Precision, Recall, F2-Score, confusion matrices, and ROC-AUC.

• To develop a **Streamlit-based user interface** for entering applicant details and obtaining loan risk predictions.

1.4 Scope of the Project

The project focuses on developing an Intelligent Loan Risk Assessment System using Machine Learning techniques. It uses a dataset containing 1,000 records and 10 columns, including applicant age, monthly income, loan amount, loan term, credit score, previous defaults, monthly debt payment, and the target variable Loan_Risk. The target variable classifies applicants into Low Risk and High Risk categories.

The scope includes data cleaning, exploratory data analysis, feature selection, DTI feature engineering, train-test splitting, handling class imbalance using SMOTE, and training three classification algorithms: Logistic Regression, Random Forest, and XGBoost. The models are compared using appropriate classification metrics, and a Streamlit interface is developed to make loan risk prediction accessible through user-provided information.

The system is intended to support preliminary loan risk assessment and demonstrate the application of Machine Learning in financial decision-making. Future enhancements may include integrating larger and more diverse datasets, incorporating additional credit history and repayment information, providing explanations for individual predictions, and deploying the application on a cloud platform. The system is a decision-support tool and is not intended to replace comprehensive financial verification or the final lending decisions of financial institutions.

CHAPTER 2

SYSTEM OVERVIEW

2.1 Existing System

Traditional loan risk assessment is generally performed by banks and financial institutions using applicant information, income verification, credit history, existing financial obligations, and repayment records. Financial institutions evaluate these factors to determine an applicant's ability to repay a loan. However, analysing multiple financial factors manually can be time-consuming, particularly when a large number of loan applications must be processed. The proposed project explores how Machine Learning can assist in analysing financial data and classifying applicants into Low Risk and High Risk categories.

2.2 Limitations of Existing System

• Manual assessment of loan applications can require considerable time and effort.

• Evaluating multiple financial factors together may be difficult and inconsistent.

• Identifying complex patterns in historical loan data through manual analysis can be challenging.

• Traditional assessment methods may not provide automated risk predictions through a simple interface.

• Imbalanced datasets can affect a model's ability to identify applicants in the minority risk category if not handled appropriately.

2.3 Proposed System

The proposed **Intelligent Loan Risk Assessment System** uses Machine Learning algorithms to predict the risk category of loan applicants based on their financial information. The project uses a dataset containing **1,000 records and 10 columns**, including age, monthly income, loan amount, loan term, credit score, previous defaults, and monthly debt payment

.

The dataset is cleaned and analysed using data preprocessing and Exploratory Data Analysis (EDA). Important features are selected, and a new feature called the **Debt-to-Income (DTI) Ratio** is created to represent the relationship between monthly debt payments and monthly income. The dataset is divided into 80% training data and 20% testing data using stratified splitting. SMOTE is applied to the training data to address class imbalance.

Three Machine Learning algorithms—**Logistic Regression, Random Forest, and XGBoost**—are trained and evaluated using Accuracy, Precision, Recall, F2-Score, confusion matrices, and ROC-AUC. A Streamlit-based user interface is developed to allow users to enter applicant information and obtain a predicted loan risk category. The system aims to support faster and more consistent preliminary loan risk assessment.

2.4 Software Requirements

• **Operating System:** Windows 10/11

• **Programming Language:** Python

• **Libraries:** Pandas, NumPy, scikit-learn, XGBoost, imbalanced-learn, Matplotlib, Seaborn, Joblib/Pickle, Streamlit

• **IDE:** Visual Studio Code / Google Colab

• **Data Format:** CSV

• **User Interface:** Streamlit

2.5 Hardware Requirements

• **Processor:** Intel Core i3 or higher (or equivalent)

• **RAM:** 8 GB or higher recommended

• **Storage:** 250 GB or higher

• **System Type:** 64-bit operating system

CHAPTER 3

SYSTEM IMPLEMENTATION

3.1 Overview of Development Tools

3.1.1 Visual Studio Code and Python

Visual Studio Code (VS Code) was used as the development environment. Unlike a notebook, VS Code handles a project made of several folders and files, which suits this project because every algorithm has its own Python file for each dataset and the user interface is a separate Streamlit application. VS Code also provides an integrated terminal for running the training scripts and the Streamlit app. A virtual environment (venv) was created inside the project folder so that the libraries used by the project stay separate from the rest of the system.

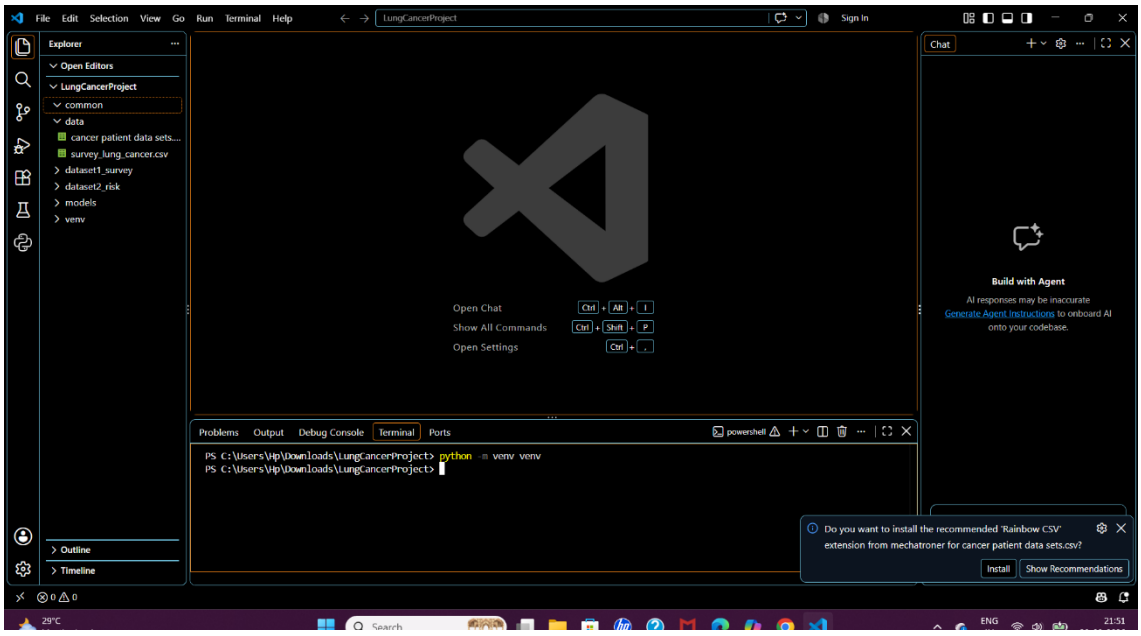


Fig 3.1: VS Code project folder structure and virtual environment setup

3.1.2 scikit-learn and Supporting Libraries

After creating the project folder and opening it in VS Code, a virtual environment was created with the command python -m venv venv and activated. On Windows PowerShell, script execution was allowed for the session with Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass. The required libraries were then installed with: pip install pandas numpy scikit-learn streamlit matplotlib seaborn joblib.

scikit-learn provides the classification algorithms, the train-test split, the evaluation metrics and permutation importance. Pandas and NumPy are used for data handling, Matplotlib and Seaborn for charts, Joblib for saving trained models and Streamlit for the web interface.

The project is organised into the following folders: data (the two CSV files), common (shared helper code for loading, cleaning, splitting and evaluating data), dataset1_survey and dataset2_risk (one Python file per algorithm), models (saved trained models) and app.py (the Streamlit interface).

3.2 Dataset Description

The dataset used in the Intelligent Loan Risk Assessment System is named loan_risk_dataset_1000_rows.csv. It contains **1,000 records and 10 columns** representing applicant details, loan information, and financial characteristics. The dataset includes applicant age, monthly income, loan amount, loan term, credit score, previous defaults, monthly debt payment, and the target variable used for risk classification.

The dataset contains two identification columns: Loan_Application_ID and Name. These columns identify individual applications and applicants, respectively, and are not used as input features for the Machine Learning models.

The target variable is Loan_Risk, which consists of two categories: **Low Risk and High Risk**. In the original dataset, 818 records are labelled High Risk, 177 records are labelled Low Risk, and 5 records have missing target values. The records with missing target values are removed during preprocessing.

The dataset is checked for missing values, duplicate records, and invalid financial values. Numerical missing values are handled using median imputation. The selected features are then used for training and testing the Machine Learning models. Since the target classes are imbalanced, SMOTE is applied to the training data to balance the class distribution.

3.3 Feature Description

The dataset contains seven initial input features selected for loan risk prediction. These features represent the applicant's demographic information, loan details, credit history, and financial obligations. During feature engineering, an additional feature called **Debt-to-Income (DTI) Ratio** is created, resulting in a final set of eight input features.

The selected features are described below:

1. Age: Represents the applicant's age in years and provides basic demographic information.

2. Monthly_Income: Represents the applicant's monthly income and helps indicate their financial capacity.

3. Loan_Amount: Represents the total amount of money requested by the applicant.

4. Loan_Term_Months: Represents the duration of the loan in months.

5. Credit_Score: Represents the applicant's credit score, which provides an indication of their creditworthiness.

6. Previous_Defaults: Represents the number of previous loan defaults recorded for the applicant.

7. Monthly_Debt_Payment: Represents the applicant's monthly debt repayment obligations.

8. DTI (Debt-to-Income Ratio): A newly engineered feature calculated using monthly debt payment and monthly income. It indicates the percentage of monthly income committed to debt payments.

The DTI Ratio is calculated using the following formula:

DTI = (Monthly_Debt_Payment / Monthly_Income) × 100

The target variable, Loan_Risk, is converted into numerical form for model training, with **Low Risk = 0 and High Risk = 1**.

The eight selected input features are used to train the Logistic Regression, Random Forest, and XGBoost models. These features help the system identify financial patterns associated with the risk category of a loan applicant.

Table 3.1: Features of the Loan Risk Dataset

No.	Feature	Type	Values	Meaning
1	**Age**	Numeric	Years	Age of the loan applicant
2	**Monthly_Income**	Numeric	Currency value	Applicant's monthly income
3	**Loan_Amount**	Numeric	Currency value	Total amount of the loan requested
4	**Loan_Term_Months**	Numeric	Number of months	Duration of the loan repayment period
5	**Credit_Score**	Numeric	Numerical score	Creditworthiness of the applicant
6	**Previous_Defaults**	Numeric	Count	Number of previous loan defaults
7	**Monthly_Debt_Payment**	Numeric	Currency value	Applicant's monthly debt repayment amount
8	**DTI**	Numeric	Percentage (%)	Ratio of monthly debt payment to monthly income
9	**Loan_Risk**	Categorical (Target)	Low / High	Predicted loan risk category of the applicant

Table 3.2: Selected Features used for Model Training

No.	Feature	Type	Values	Meaning
1	**Age**	Numeric	Years	Age of the loan applicant
2	**Monthly_Income**	Numeric	Currency value	Applicant's monthly income
3	**Loan_Amount**	Numeric	Currency value	Total amount of the loan requested
4	**Loan_Term_Months**	Numeric	Number of months	Duration of the loan repayment period
5	**Credit_Score**	Numeric	Numerical score	Creditworthiness of the applicant
6	**Previous_Defaults**	Numeric	Count	Number of previous loan defaults
7	**Monthly_Debt_Payment**	Numeric	Currency value	Applicant's monthly debt payment amount
8	**DTI**	Numeric	Percentage (%)	Ratio of monthly debt payment to monthly income

3.4 Running the Project

The project is run in two stages. First, each algorithm file is run once to train its model and save it in the models folder. Then the Streamlit application is started, which loads the saved models. After opening the project folder in VS Code and opening a terminal, the following commands are used:

```
python -m venv venv
venv\Scripts\activate
pip install pandas numpy scikit-learn streamlit matplotlib seaborn joblib

python dataset1_survey/logistic_regression.py     (repeat for the other five
algorithms)
python dataset2_risk/logistic_regression.py       (repeat for the other five
algorithms)

streamlit run app.py
```

Each training script prints the number of training and testing records, the accuracy, precision, recall and F1-score, and a classification report, saves the model in the models folder and shows the confusion matrix; closing the chart window lets the script finish. The command streamlit run app.py opens the application in the browser at http://localhost:8501.

CHAPTER 4

SYSTEM ARCHITECTURE

The system works in seven stages, from collecting the loan dataset to presenting predictions through the user interface. It uses Machine Learning to classify loan applicants into Low Risk and High Risk categories based on their financial information.

1. Data Collection: The loan risk dataset containing 1,000 records and 10 columns is loaded from the CSV file.

2. Data Preprocessing: Missing values, duplicate records, and invalid financial values are checked and handled. The identification columns are excluded from model training, and the target variable is encoded into numerical values.

3. Exploratory Data Analysis (EDA): The dataset is analysed using statistical summaries and visualizations to understand feature distributions, identify patterns, and examine relationships between financial variables and loan risk.

4. Feature Selection and Engineering: Seven important features are selected: Age, Monthly_Income, Loan_Amount, Loan_Term_Months, Credit_Score, Previous_Defaults, and Monthly_Debt_Payment. A new feature, Debt-to-Income (DTI) Ratio, is created, resulting in eight input features.

5. Train-Test Split and SMOTE: The dataset is divided into 80% training data and 20% testing data using stratified splitting. SMOTE is applied only to the training data to address class imbalance.

6. Model Training and Evaluation: Three Machine Learning algorithms—Logistic Regression, Random Forest, and XGBoost—are trained using the prepared data. Their performance is compared using Accuracy, Precision, Recall, F2-Score, confusion matrices, and ROC-AUC.

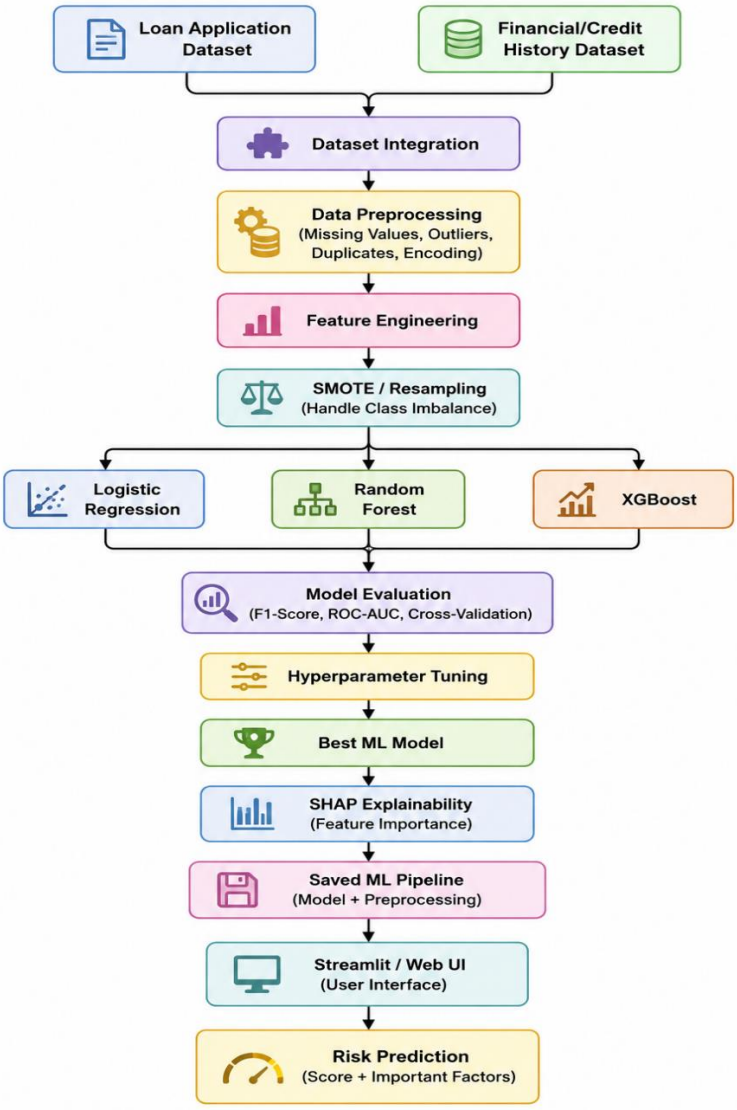
7. Streamlit Interface: The user enters the applicant's financial details through the Streamlit interface and receives a predicted loan risk category. The interface provides a simple way to interact with the trained model.

Figure 4.1 illustrates the overall architecture of the Intelligent Loan Risk Assessment System, consisting of three stages: input, processing, and output.

The **input stage** uses the loan dataset containing 1,000 records and applicant financial information.

The **processing stage** includes data preprocessing, feature selection, DTI feature engineering, stratified train-test splitting, SMOTE, and training three models: Logistic Regression, Random Forest, and XGBoost.

The **output stage** includes model evaluation and a Streamlit interface that predicts whether an applicant belongs to the Low Risk or High Risk category.



- **common/data_utils.py:** loads and cleans both datasets, performs the 80/20 split, calculates the evaluation metrics and saves each trained model.
- **dataset1_survey and dataset2_risk:** one Python file for each of the six algorithms; each file trains one model and saves it.
- **models:** the twelve saved models, each file holding the model, its metrics, its confusion matrix and its feature importance.
- **app.py:** the Streamlit interface that loads the saved models and provides the Predict, Risk Factor Analysis and Model Performance tabs.

CHAPTER 5

DATA PREPROCESSING

5.1 Introduction

Data preprocessing is an important step in Machine Learning because raw datasets may contain missing values, duplicate records, and invalid values. In this project, Pandas and NumPy were used to clean and prepare the loan risk dataset, while scikit-learn was used for imputation, data splitting, and feature scaling. The dataset was prepared for training three Machine Learning models: Logistic Regression, Random Forest, and XGBoost.

5.2 Handling Missing Values

The dataset was checked for missing values using Pandas. Missing values in numerical features were handled using median imputation. Records with missing values in the target column Loan_Risk were removed because the target category is required for supervised model training. After preprocessing, the dataset was checked again to verify the missing values.

5.3 Data Cleaning

Duplicate records were identified using the duplicated() function and removed using drop_duplicates(). The Name column was standardized by removing unnecessary spaces and converting names to title case. The identification columns Loan_Application_ID and Name were excluded from model training because they do not represent the financial characteristics required for loan risk prediction.

5.4 Outlier Detection and Range Validation

The dataset was checked for invalid values in important numerical features. Age values below 18 or above 80 were treated as missing values. Credit scores below 300 or above 900 were also treated as invalid. Negative values in Previous_Defaults were replaced with missing values. These values were handled using median imputation to maintain a consistent dataset for model training.

During feature engineering, non-positive values in Monthly_Income and negative values in Monthly_Debt_Payment were also handled to avoid invalid DTI calculations.

5.5 Feature Engineering and Encoding

Seven initial features were selected: Age, Monthly_Income, Loan_Amount, Loan_Term_Months, Credit_Score, Previous_Defaults, and Monthly_Debt_Payment.

A new feature called **Debt-to-Income (DTI) Ratio** was created to represent the relationship between monthly debt payment and monthly income.

DTI = (Monthly_Debt_Payment / Monthly_Income) × 100

Missing DTI values were filled using the median. This feature increased the final number of selected input features from seven to eight.

The target variable Loan_Risk was encoded as **Low Risk = 0** and **High Risk = 1**. The identification columns were excluded from the model input, leaving eight features for training.

5.6 Data Visualization

Exploratory Data Analysis was performed using Matplotlib and Seaborn. Count plots were used to examine the distribution of loan risk categories and loan terms. Histograms were used to understand monthly income distribution. Box plots compared credit scores and monthly income across risk categories, while a scatter plot examined the relationship between monthly income and loan amount. These visualizations helped identify patterns and understand the dataset before model training.

5.7 Train-Test Split and Feature Scaling

The dataset was divided into **80% training data and 20% testing data** using train_test_split(). Stratified splitting was applied to maintain the proportion of Low Risk and High Risk classes in both sets. The random state was set to 42 to ensure reproducibility.

Median imputation was applied after splitting. The imputer was fitted on the training data and then used to transform the testing data. **SMOTE (Synthetic Minority Over-sampling Technique)** was applied only to the training data to address class imbalance, while the test data remained unchanged.

StandardScaler was used to standardize the input features for Logistic Regression. The scaler was fitted on the SMOTE-balanced training data and then applied to the testing data. Random Forest and XGBoost were trained using the prepared, unscaled features.

5.8 Summary

The preprocessing stage prepared the loan risk dataset for Machine Learning by removing duplicate records, handling missing and invalid values, excluding identification columns, and encoding the target variable. Feature engineering introduced the DTI Ratio, resulting in eight input features. Stratified train-test splitting, median imputation, SMOTE, and feature scaling were applied where required. The prepared dataset was then used to train Logistic Regression, Random Forest, and XGBoost models.

Table 5.1: Preprocessing Steps Applied to the Loan Risk Dataset

Step	Technique Applied	Purpose
Missing values	Median imputation	Handle missing numerical values
Missing target values	Removal of affected records	Ensure valid target labels
Duplicate records	drop_duplicates()	Remove repeated records
Name cleaning	Strip spaces and title case	Standardize name formatting
Invalid Age values	Values outside 18–80 treated as missing	Improve data consistency
Invalid Credit Scores	Values outside 300–900 treated as missing	Handle invalid credit scores
Previous Defaults	Negative values treated as missing	Remove invalid default counts
Feature selection	Seven initial financial features	Select relevant input variables
Feature engineering	DTI Ratio creation	Add a debt-burden feature
Target encoding	Low = 0, High = 1	Convert target labels to numbers
Train-test split	80% training, 20% testing	Separate training and evaluation data
Class balancing	SMOTE on training data only	Address class imbalance
Feature scaling	StandardScaler for Logistic Regression	Standardize feature values

CHAPTER 6

METHODOLOGY AND ALGORITHMS

6.1 Introduction

The Intelligent Loan Risk Assessment System is a supervised Machine Learning classification project that predicts whether a loan applicant belongs to the Low Risk or High Risk category. The project uses a dataset containing applicant information and financial features to train and evaluate three Machine Learning algorithms: Logistic Regression, Random Forest, and XGBoost.

The methodology includes data preprocessing, exploratory data analysis, feature selection, feature engineering, train-test splitting, class balancing, model training, and model evaluation. The dataset is divided into 80% training data and 20% testing data using stratified splitting. SMOTE is applied to the training data to address class imbalance. The trained models are compared using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

6.2 Logistic Regression

Working

Logistic Regression is a supervised classification algorithm that estimates the probability of an applicant belonging to a particular risk category. It uses a logistic function to convert a weighted combination of input features into a probability between 0 and 1. Based on the predicted probability, the applicant is classified as Low Risk or High Risk.

Advantages

- Simple and computationally efficient.
- Suitable for binary classification problems.
- Provides probability estimates for predictions.
- Helps understand relationships between input features and the predicted class.

Application in Project

Logistic Regression was implemented using LogisticRegression from scikit-learn with max_iter = 2000 and random_state = 42. The model was trained using the scaled training data obtained after applying SMOTE. The selected eight features were used to predict the Loan_Risk target variable.

The model achieved **88.94% accuracy**, with a Precision of 97.97%, Recall of 88.41%, F1-Score of 92.95%, and ROC-AUC of 93.19%.

```
models = {

    "Logistic Regression": LogisticRegression(
        max_iter=2000,
        random_state=42
    ),
```

6.3 Random Forest

Working

Random Forest is an ensemble Machine Learning algorithm that combines multiple decision trees to make predictions. Each tree learns patterns from the training data, and the final classification is determined by combining the predictions of the individual trees. This approach helps capture complex relationships between financial features and loan risk.

Advantages

- Combines multiple decision trees to improve prediction stability.
- Can capture complex relationships between features.
- Does not require feature scaling.
- Provides feature importance values to identify influential input variables

Application in Project

Random Forest was implemented using RandomForestClassifier from scikit-learn. The model was configured with **300 decision trees**, a maximum depth of 8, a minimum of 3 samples per leaf, and random_state = 42. The model was trained using the SMOTE-balanced training data.

Random Forest achieved **91.96% accuracy**, with a Precision of 98.05%, Recall of 92.07%, F1-Score of 94.97%, and ROC-AUC of 96.41%. Among the three models, Random Forest achieved the highest Accuracy, Precision, Recall, and F1-Score.

Fig. 6.2: Training the Random Forest Model

```
"Random Forest": RandomForestClassifier(
    n_estimators=300,
    max_depth=8,
    min_samples_leaf=3,
    random_state=42,
    n_jobs=-1
```

6.4 XGBoost

Working

XGBoost (Extreme Gradient Boosting) is an ensemble learning algorithm that builds decision trees sequentially. Each new tree attempts to correct errors made by the previous trees. By combining the predictions of multiple trees, XGBoost learns patterns in the financial data and predicts the risk category of an applicant.

Advantages

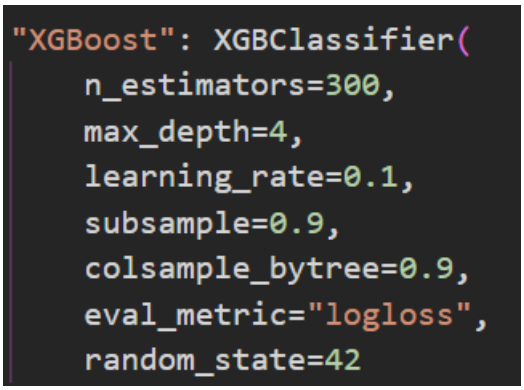
- Captures complex relationships between financial features.
- Uses boosting to improve predictive performance.
- Supports regularization to help control model complexity.
- Provides feature importance information.

Application in Project

XGBoost was implemented using XGBClassifier. The model was configured with **300 estimators**, a maximum depth of 4, a learning rate of 0.1, subsample = 0.9, colsample_bytree = 0.9, and random_state = 42. The evaluation metric was set to logloss.

The model was trained using the SMOTE-balanced training data and the selected eight features. XGBoost achieved **89.95% accuracy**, with a Precision of 98.00%, Recall of 89.63%, F1-Score of 93.63%, and ROC-AUC of 97.04%. It achieved the highest ROC-AUC among the three models.

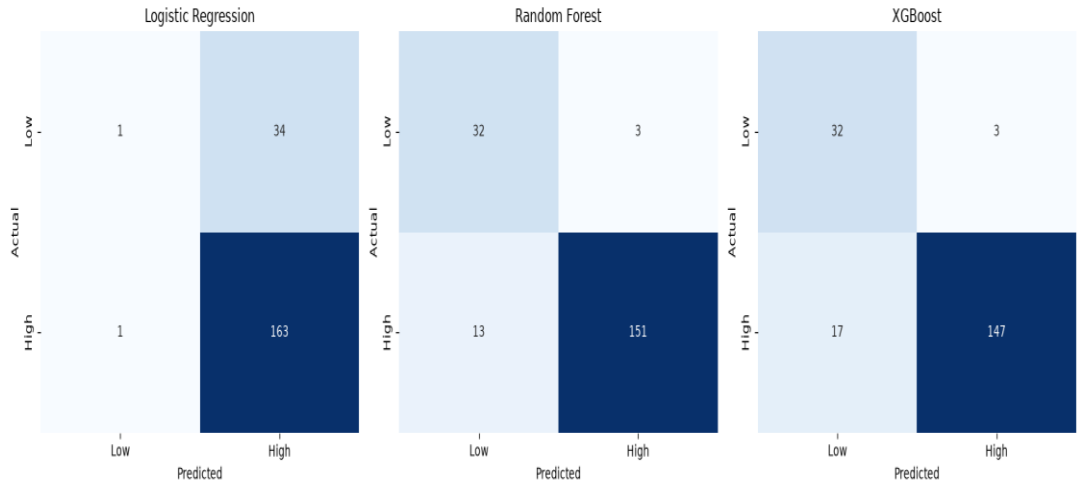
Fig. 6.3: Training the XGBoost Model

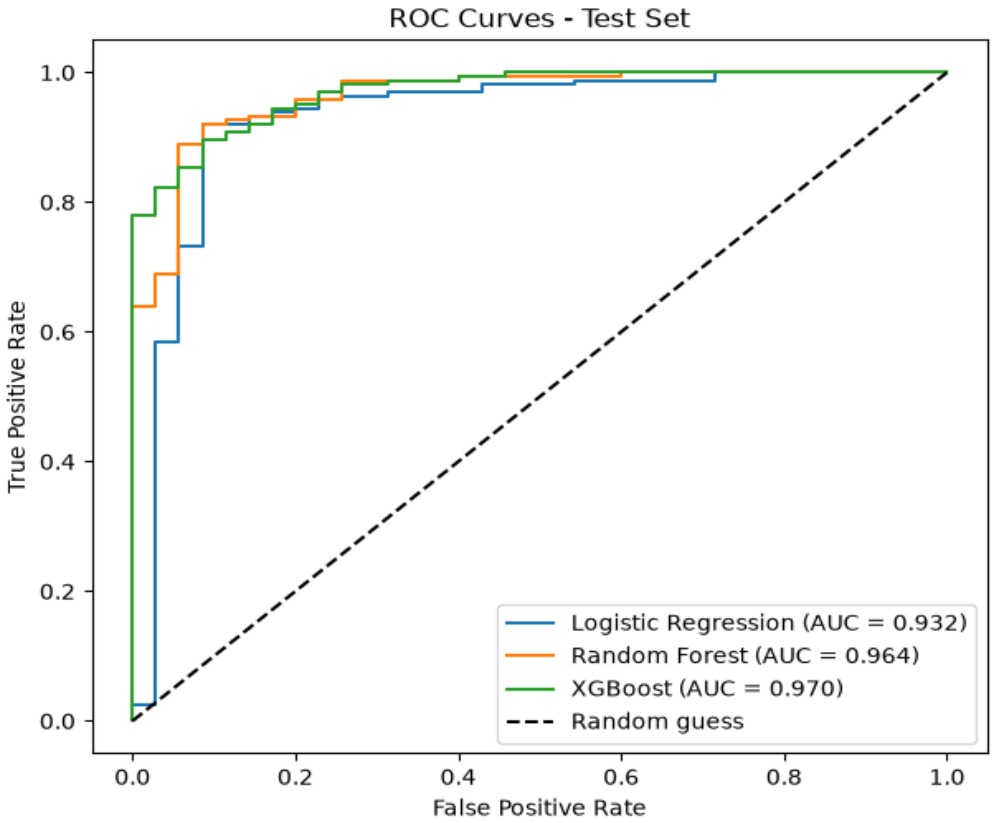


6.5 Model Evaluation Metrics

The trained models were evaluated using several classification metrics to understand their ability to predict Low Risk and High Risk applicants.

- **Accuracy:** Measures the proportion of all predictions that are correct.
- **Precision:** Measures the proportion of applicants predicted as High Risk who actually belong to the High Risk category.
- **Recall:** Measures the proportion of actual High Risk applicants correctly identified by the model.
- **F1-Score:** Represents the harmonic mean of Precision and Recall.
- **ROC-AUC:** Measures the ability of a model to distinguish between Low Risk and High Risk applicants across different classification thresholds.
- **Confusion Matrix:** Displays the number of correct and incorrect predictions for each risk category





	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.8894	0.9797	0.8841	0.9295	0.9319
Random Forest	0.9196	0.9805	0.9207	0.9497	0.9641
XGBoost	0.8995	0.9800	0.8963	0.9363	0.9704

6.6 Algorithm Configuration

Table 6.1: Configuration of the Three Machine Learning Algorithms

Algorithm	Scikit-learn Class	Main Settings	Feature Scaling
Logistic Regression	LogisticRegression	max_iter = 2000, random_state = 42	StandardScaler
Random Forest	RandomForestClassifier	n_estimators = 300, max_depth = 8, min_samples_leaf = 3, random_state = 42	Not required
XGBoost	XGBClassifier	n_estimators = 300, max_depth = 4, learning_rate = 0.1, subsample = 0.9, colsample_bytree = 0.9	Not required

The three algorithms were trained using the same selected features and the SMOTE-balanced training dataset. StandardScaler was used for Logistic Regression, while Random Forest and XGBoost were trained using the unscaled features.

6.7 Training and Saving Procedure

The model development process follows these steps:

Step 1: Dataset Loading

The prepared loan risk dataset is loaded using Pandas.

Step 2: Feature Selection

Eight input features are selected, including the engineered DTI Ratio. The target variable Loan_Risk is encoded as Low = 0 and High = 1.

Step 3: Train-Test Split

The dataset is divided into 80% training data and 20% testing data using stratified splitting, with random_state = 42.

Step 4: Missing Value Imputation

A SimpleImputer using the median strategy is fitted on the training data and applied to the testing data.

Step 5: SMOTE Application

SMOTE is applied to the training data to balance the Low Risk and High Risk classes.

Step 6: Feature Scaling

StandardScaler is fitted on the SMOTE-balanced training features and applied to the test features for Logistic Regression.

Step 7: Model Training

Logistic Regression, Random Forest, and XGBoost are trained using the prepared training data.

Step 8: Model Evaluation

The models are evaluated on the test data using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and confusion matrices.

Step 9: Feature Importance Analysis

Feature importance values are extracted from the trained Random Forest and XGBoost models. Logistic Regression coefficients are also examined to understand the contribution of the selected features.

Step 10: Model Saving

The trained Random Forest model, median imputer, and selected feature names are saved using Joblib for later use.

The saved model and preprocessing files can then be loaded by the Streamlit application to generate predictions for new loan applications.

CHAPTER 7
RESULTS AND ANALYSIS

7.1 Introduction

This chapter presents the experimental results of the Intelligent Loan Risk Assessment System. Three Machine Learning algorithms—Logistic Regression, Random Forest, and XGBoost—were trained and evaluated to predict whether a loan applicant belongs to the Low Risk or High Risk category. Their performance was compared using Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Confusion matrices and feature importance visualizations were also generated to analyse the prediction results and understand the contribution of different financial features.

7.2 Experimental Setup

The loan risk dataset was preprocessed, and the target variable was encoded as Low Risk = 0 and High Risk = 1. Eight input features were selected, including the engineered Debt-to-Income (DTI) Ratio.

The dataset was divided into 80% training data and 20% testing data using stratified splitting with random_state = 42. After removing records with missing target values, the dataset contained 995 records, of which 796 were used for training and 199 for testing.

Median imputation was used to handle missing numerical values. SMOTE was applied to the training data, increasing its size from 796 to 1,308 records and balancing the two risk classes. StandardScaler was applied to the training and testing features used by Logistic Regression.

The three models were trained and evaluated on the same held-out test dataset. Accuracy, Precision, Recall, F1-Score, and ROC-AUC were calculated to compare their performance. A separate validation set and cross-validation were not used in this experiment.

7.3 Logistic Regression Results

Logistic Regression was used as a baseline classification algorithm for predicting loan risk. It estimates the probability of an applicant belonging to the High Risk category based on the selected financial features.

The model achieved an accuracy of 88.94%, a Precision of 97.97%, a Recall of 88.41%, an F1-Score of 92.95%, and a ROC-AUC of 93.19%.

The results indicate that Logistic Regression performed well in identifying loan-risk categories. Its high Precision shows that most applicants predicted as High Risk belonged to that category. However, its Recall was lower than that of Random Forest, indicating that it missed a greater proportion of actual High Risk applicants.

7.4 Random Forest Results

Random Forest was implemented using 300 decision trees, a maximum depth of 8, and a minimum of three samples per leaf. The algorithm combines predictions from multiple decision trees to identify patterns in the financial features.

Random Forest achieved an accuracy of 91.96%, a Precision of 98.05%, a Recall of 92.07%, an F1-Score of 94.97%, and a ROC-AUC of 96.41%.

Among the three models, Random Forest achieved the highest Accuracy, Precision, Recall, and F1-Score. These results indicate that it provided the strongest overall classification performance on the test dataset. Its combination of high Precision and Recall makes it a promising model for identifying high-risk loan applicants.

7.5 XGBoost Results

XGBoost was implemented using 300 estimators, a maximum tree depth of 4, and a learning rate of 0.1. It builds decision trees sequentially to improve classification performance.

The model achieved an accuracy of 89.95%, a Precision of 98.00%, a Recall of 89.63%, an F1-Score of 93.63%, and a ROC-AUC of 97.04%.

XGBoost achieved the highest ROC-AUC among the three models, indicating that it demonstrated the strongest ability to distinguish between Low Risk and High Risk applicants across different classification thresholds. However, its Accuracy, Recall, and F1-Score were lower than those of Random Forest.

7.6 Comparison of Models

The performance of the three Machine Learning models was compared using the evaluation metrics obtained from the test dataset.

Table 7.1: Performance Comparison of Machine Learning Models

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	88.94%	97.97%	88.41%	92.95%	93.19%
Random Forest	91.96%	98.05%	92.07%	94.97%	96.41%
XGBoost	89.95%	98.00%	89.63%	93.63%	97.04%

The comparison shows that Random Forest achieved the highest Accuracy, Precision, Recall, and F1-Score. XGBoost achieved the highest ROC-AUC, while Logistic Regression provided a competitive baseline result.

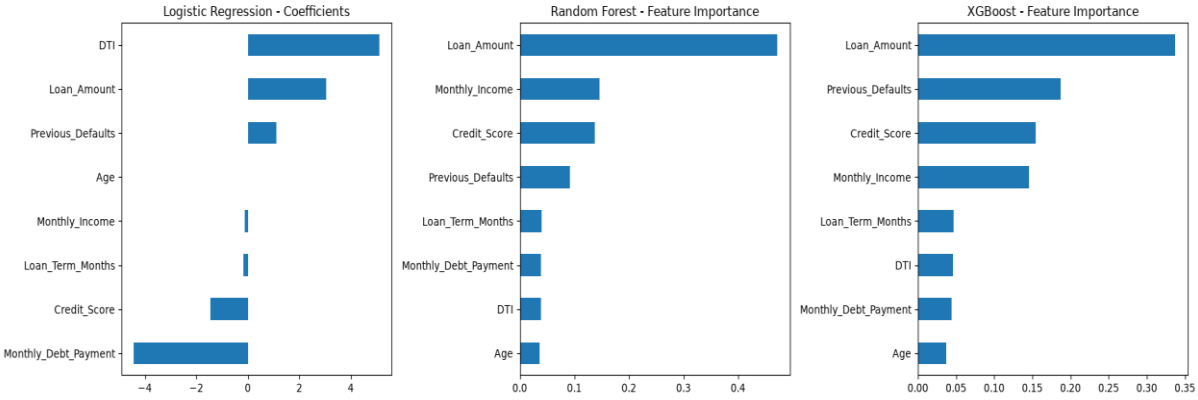
Overall, Random Forest was the strongest model based on the main classification metrics, whereas XGBoost demonstrated the best ROC-AUC performance.

7.7 Feature Importance Analysis

Feature importance analysis was performed to understand how the selected financial features contributed to loan risk prediction. The notebook generated feature importance visualizations using Logistic Regression coefficients, Random Forest feature importance values, and XGBoost feature importance values.

The analysis considered eight input features: Age, Monthly_Income, Loan_Amount, Loan_Term_Months, Credit_Score, Previous_Defaults, Monthly_Debt_Payment, and DTI.

These visualizations help identify which features the trained models rely on most when classifying loan applicants. The relative importance of individual features may differ between algorithms because each model learns patterns in a different way.



7.8 Key Findings

The following findings were obtained from the experimental results:

- Random Forest achieved the highest Accuracy of 91.96% and the highest F1-Score of 94.97% among the three models.
- Random Forest also achieved the highest Precision of 98.05% and Recall of 92.07%.
- XGBoost achieved the highest ROC-AUC of 97.04%, demonstrating strong class discrimination across different thresholds.
- Logistic Regression achieved an Accuracy of 88.94%, providing a useful baseline for comparison.
- SMOTE balanced the training classes from 654 High Risk and 142 Low Risk records to 654 records in each class.
- Feature importance visualizations provided a way to examine the contribution of the eight selected financial features.
- Based on the classification metrics, Random Forest was the strongest overall model in this experiment.

7.9 Summary

This chapter presented the experimental results and performance comparison of Logistic Regression, Random Forest, and XGBoost for loan risk prediction. Random Forest achieved the highest Accuracy, Precision, Recall, and F1-Score, while XGBoost achieved the highest ROC-AUC. Feature importance visualizations were also generated to examine the contribution of the selected financial features. Based on these results, Random Forest demonstrated the strongest overall performance on the held-out test dataset.

7.10 Discussion

Model Performance: Random Forest achieved the highest accuracy of 91.96%, followed by XGBoost at 89.95% and Logistic Regression at 88.94%. Random Forest performed better overall because it combines multiple decision trees to identify patterns in the financial data.

Precision and Recall: All three models achieved high Precision values, with Random Forest reaching 98.05%. Random Forest also achieved the highest Recall of 92.07%, indicating that it identified a larger proportion of actual High Risk applicants than the other models.

ROC-AUC Comparison: XGBoost achieved the highest ROC-AUC of 97.04%, followed by Random Forest at 96.41% and Logistic Regression at 93.19%. This indicates that XGBoost demonstrated the strongest ability to distinguish between Low Risk and High Risk applicants across different classification thresholds.

Feature Engineering: The Debt-to-Income (DTI) Ratio was created to represent the proportion of monthly income committed to debt payments. This additional feature provides the models with useful information about applicants' financial obligations.

Class Imbalance: SMOTE was applied to the training data to balance the Low Risk and High Risk classes. This helped provide a balanced training dataset while keeping the test data unchanged for evaluation.

Practical Application: The Streamlit interface provides a simple way to enter applicant financial information and obtain a loan risk prediction. The system demonstrates how Machine Learning can support preliminary loan risk assessment, although its predictions should not replace comprehensive financial verification.

7.11 Limitations of the Study

• The dataset contains 1,000 original records, and records with missing target values were removed before model training.

• The dataset contains a limited number of applicant and financial features, which may not represent every factor involved in actual lending decisions.

• A single stratified 80/20 train-test split was used; cross-validation was not performed.

• Extensive hyperparameter tuning was not performed, so better results may be possible with further optimization.

• SMOTE creates synthetic training samples, which may not fully represent the financial characteristics of real loan applicants.

• The system's predictions depend on the quality and representativeness of the training dataset.

• The model has not been validated using real-world lending outcomes and should be treated as an educational decision-support system rather than an autonomous loan approval mechanism.

7.12 Key Findings

• Random Forest achieved the highest accuracy of 91.96%, followed by XGBoost at 89.95% and Logistic Regression at 88.94%.

• Random Forest achieved the highest Precision (98.05%), Recall (92.07%), and F1-Score (94.97%) among the three models.

• XGBoost achieved the highest ROC-AUC of 97.04%, indicating strong discrimination between the two loan risk categories.

• The DTI Ratio was introduced as an additional feature to represent the relationship between monthly debt payments and monthly income.

• SMOTE was applied only to the training data to address class imbalance without modifying the original test data.

• The Streamlit interface provides an accessible way to obtain Low Risk or High Risk predictions from applicant financial information.

CHAPTER 8

OUTPUT SCREENS

CHAPTER 9

OUTPUT SCREENS

This chapter presents the output screen of the Intelligent Loan Risk Assessment System developed using Streamlit. The application provides a user-friendly interface for entering applicant financial details and predicting the applicant's loan risk category. Based on the information provided, the trained machine learning model classifies the applicant as either Low Risk or High Risk. The output screen demonstrates the interaction between the user and the prediction system.

Fig. 8.1: Streamlit Application – Loan Risk Prediction

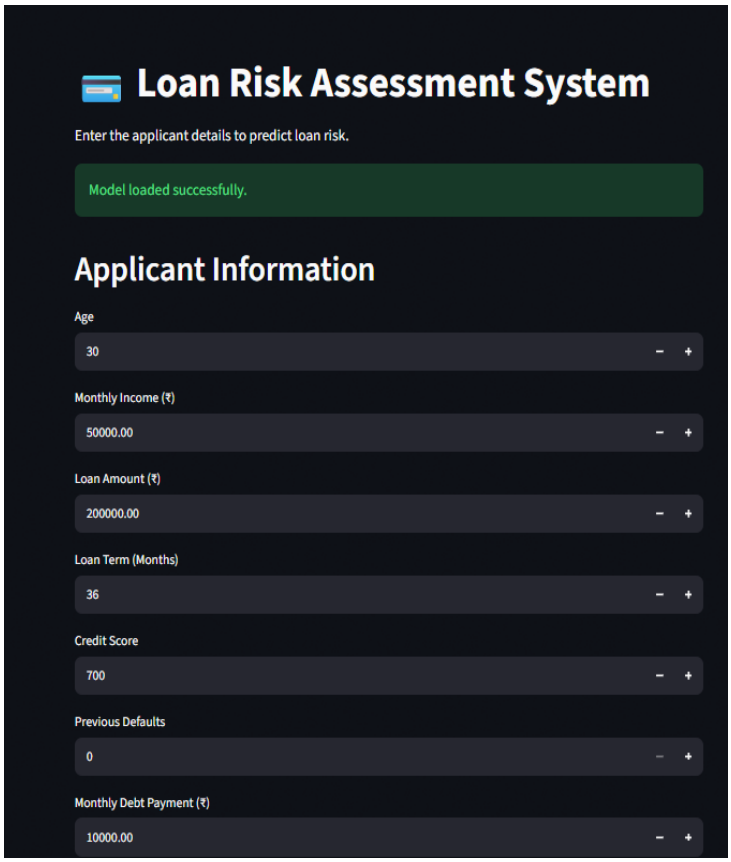


Fig. 8.2: Loan Risk Prediction Result

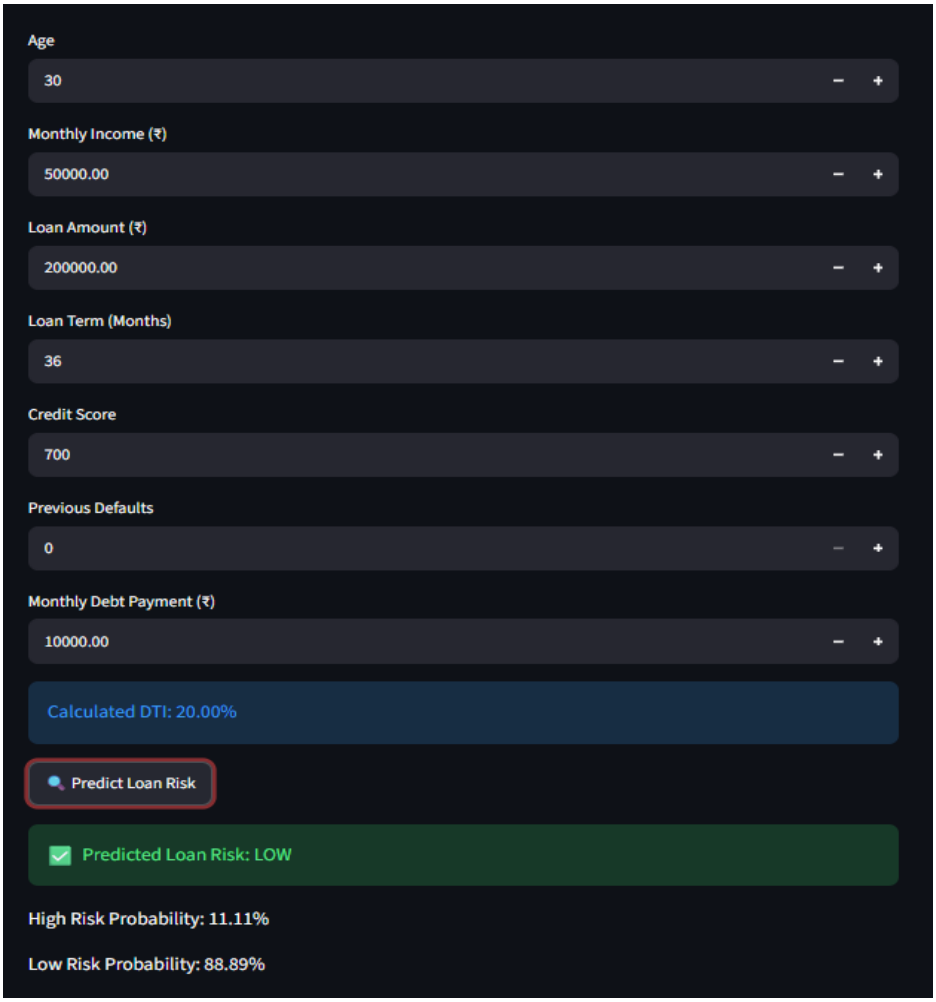


Fig 8.3 Monthly Income Vs Loan Amount

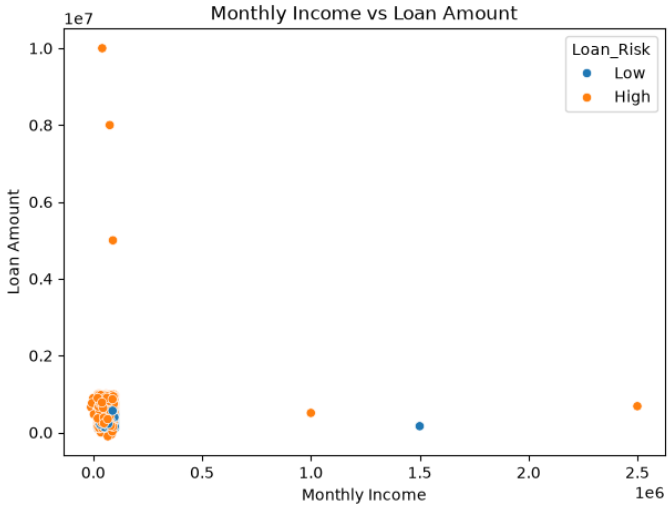
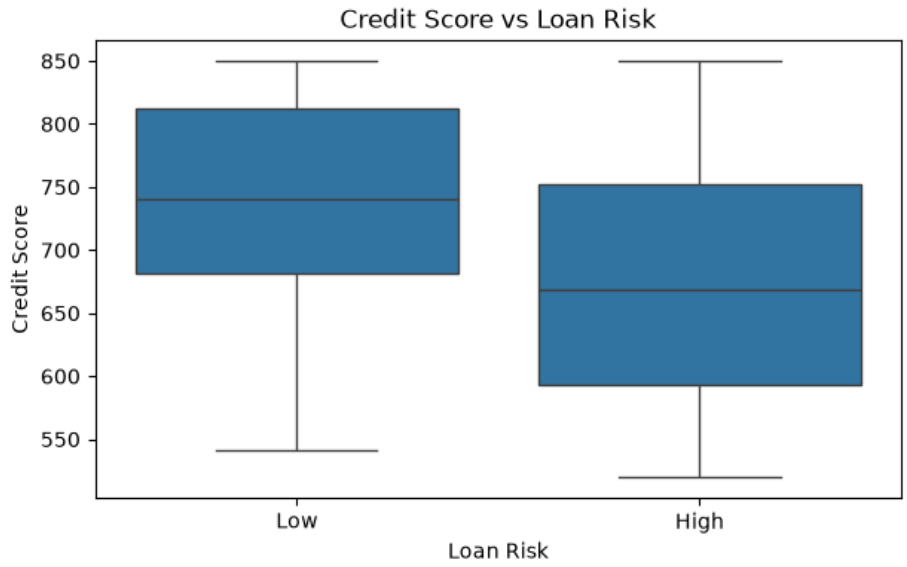


Fig 8.4 Credit Score Distribution



CONCLUSION

This project addressed the need for an efficient and systematic approach to loan risk assessment by applying machine learning techniques to a loan risk dataset. The dataset was cleaned and preprocessed by handling missing and invalid values, selecting relevant financial features, and creating a Debt-to-Income Ratio (DTI) feature. The data was divided into training and testing sets using a stratified 80/20 split. To address class imbalance, SMOTE was applied to the training data. Three machine learning algorithms—Logistic Regression, Random Forest, and XGBoost—were trained and evaluated using accuracy, precision, recall, F1-score, and ROC-AUC.

Among the three models, Random Forest achieved the highest accuracy of 91.96%, with a precision of 98.05%, recall of 92.07%, and F1-score of 94.97%. XGBoost achieved the highest ROC-AUC of 97.04%, indicating strong ability to distinguish between the two loan risk categories. Logistic Regression provided a baseline for comparison. These results demonstrate that machine learning models can identify patterns in applicant financial information and support the classification of applicants into Low Risk and High Risk categories.

The Streamlit interface provides a simple way to enter applicant details and obtain a predicted loan risk category. The system can support preliminary loan risk assessment and help users understand how machine learning can be applied in financial decision-making. However, the dataset size, selected features, and limited validation may affect the generalizability of the results. The system should therefore be used as a decision-support tool rather than as the sole basis for loan approval. Its reliability could be improved through larger and more representative real-world datasets, extensive hyperparameter tuning, cross-validation, and further evaluation in practical lending environments.

CHAPTER 10

FUTURE SCOPE

- Using larger, more diverse, and real-world loan datasets to improve the reliability and generalizability of loan risk predictions.
- Comparing additional machine learning algorithms and advanced ensemble techniques, along with cross-validation and hyperparameter tuning, to improve model performance.
- Integrating additional financial features, such as repayment history, existing loans, employment stability, and credit utilization, to provide a more comprehensive assessment of loan risk.
- Deploying the Streamlit application online and extending it into a web or mobile application to make the system more accessible.
- Integrating the system with authorized banking databases and credit information systems, where permitted, to support assessment using updated financial information.
- Adding Explainable AI techniques, such as SHAP, to identify the factors influencing individual predictions and improve transparency.
- Developing an automated pipeline or dashboard for model retraining, performance monitoring, and data quality checks to maintain the reliability of the system over time.

CHAPTER 11

REFERENCES

[1] Loan Risk Dataset, loan_risk_dataset_1000_rows.csv, dataset used for training and evaluating the machine learning models.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[3] J. H. Friedman, "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[4] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[9] Streamlit Inc., "Streamlit Documentation." Available: https://docs.streamlit.io/

[10] Pandas Development Team, "pandas Documentation." Available: https://pandas.pydata.org/docs/

[11] XGBoost Developers, "XGBoost Documentation." Available: https://xgboost.readthedocs.io/